Week-1

1.Finding Complexity using Counter Method

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n) {
 int i= 1;
 int s =1;
while(s <= n) {
 i++;
 s += i;
 }
 }
```

Input: A positive Integer n

Output: Print the value of the counter variable

For example:

Input	Result
9	12

Source code:

```
#include <stdio.h>
int main()
{
   int n;
   scanf("%d", &n);
   int counter = 0;
   int i = 1;
   int s = 1;
```

```
    counter += 2;
    while (1)
    {
        counter++;
        if (s > n)
            break;
        i++;
        counter++;
        s += i;
        counter++;
    }
    printf("%d", counter);
    return 0;
}
```

Output:

Input	Expected	Got
9	12	12
4	9	9

2. Finding Complexity using Counter method

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n){
  if(n==1){
    printf("*");
  }
  else{
   for(int i=1; i<=n; i++) {
     for(int j=1; j<=n; j++){
       printf("*");
       printf("*");
       break;
     }
   }
 }
}
```

Input:

A positive Integer n

Output:

Print the value of the counter variable

Source code:

```
#include <stdio.h>
int main()
{
  int n;
  scanf("%d", &n);
  int counter = 0;
  if (n == 1)
  {
```

```
    counter++;
  }
  else
  {
    counter++;
    for (int i = 1; i <= n; i++, counter++)
    {
      counter++;
      for (int j = 1; j <= n; j++, counter++)
      {
        counter++;
        counter++;
        break;
      }
      counter++;
    }
    counter++;
  }
  printf("%d", counter);
  return 0;
}
```

Output:

Input	Expected	Got
2	12	12
1000	5002	5002
143	717	717

3. Finding Complexity using Counter method

Convert the following algorithm into a program and find its time complexity using counter method.

```
Factor(num) {
  for (i = 1; i <= num;++i){
   if (num % i== 0){
      printf("%d ", i);
    }
  }
}
```

Input:

A positive Integer n

Output:

Print the value of the counter variable

Source code:

```
#include <stdio.h>

int main(){

  int num, i, counter = 0;

  scanf("%d", &num);

  for (i = 1; i <= num; i++){

    counter++;

    counter++;

    if (num % i == 0){

      counter++;

    }

  }

  counter++;

  printf("%d", counter);

  return 0;

}
```

Output:

Input	Expected	Got
12	31	31
25	54	54
4	12	12

4. Finding Complexity using Counter Method

Convert the following algorithm into a program and find its time

complexity using counter method.

```
void function(int n)
{
  int c= 0;
  for(int i=n/2; i<n; i++)
    for(int j=1; j<n; j = 2 * j)
      for(int k=1; k<n; k = k * 2)
        c++;
}
```

Note: No need of counter increment for declarations and scanf() and printf() statements. count variable

Input:
A positive Integer n
Output:
Print the value of the counter variable

Source code:

```
#include <stdio.h>

int main()

{

  int n;

  scanf("%d", &n);
```

```
  int c = 0;
  c++;
  for (int i = n / 2; i < n; i++)
  {
    c++;
    for (int j = 1; j < n; j = 2 * j)
    {
      c++;
      for (int k = 1; k < n; k = k * 2)
      {
        c++;
        c++;
      }
      c++;
    }
    c++;
  }
  c++;
  printf("%d", c);
  return 0;
}
```

Output:

Input	Expected	Got
4	30	30
10	212	212

5. Finding Complexity using counter method

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
  int rev = 0, remainder;
  while (n != 0)
   {
      remainder = n % 10;
      rev = rev * 10 + remainder;
      n/= 10;

   }
print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and printf() statements. count variable

Input:
A positive Integer n
Output:
Print the value of the counter variable

Source code:

```
#include <stdio.h>

int main()

{

   int n;

   scanf("%d", &n);

   int counter = 0;

   counter++;

   int rev = 0;

   counter++;

   int remainder;
```

```
    while (n != 0)
    {
        counter++;
        remainder = n % 10;
        counter++;
        rev = rev * 10 + remainder;
        counter++;
        n /= 10;
        counter++;
    }
    counter++;
    printf("%d", counter);
    return 0;
}
```

Output:

Input	Expected	Got
12	11	11
1234	19	19